Modelli AMD Token Factory sul nodo gnode01 (gpu_H100)

2× NVIDIA H100 da ~96 GB collegate via NVLink (~190 GB di VRAM) · 2× AMD EPYC 9124 (32 core) · 512 GB DDR5. Stime aggiornate a ottobre 2026.

Modello	Parametri (totali / attivi)	Su gnode01	Come farlo girare
MinerU2.5-Pro	1,2B, denso	**Comodo**	1 GPU, ne usa una minima parte
MiniCPM5-2B	2,5B, denso	**Comodo**	1 GPU
Qwen3.8-27B	27B, denso	**Comodo**	BF16 su 1 GPU (~56 GB, restano ~40 GB per la KV cache). Oppure un'istanza per GPU per raddoppiare il throughput
Gemma-4-31B-IT	30,7B, denso	**Comodo**	BF16 su 1 GPU (~62 GB), oppure un'istanza per GPU
Qwen3.8-Flash-Next	125B / ~6B MoE (+ N-gram 51B)	**Sì**	FP8 con tensor parallel su 2 GPU: ~125 GB di pesi, restano ~60 GB per la KV cache. La tabella N-gram (~100 GB) va nei 512 GB di RAM; va verificato che vLLM la gestisca
DeepSeek-V4-Flash	284B / 13B MoE	**Al limite**	Checkpoint FP4+FP8 da ~160 GB: entra solo con contesto corto e solo se vLLM supporta FP4 su Hopper. In alternativa GGUF Q4 con llama.cpp, scaricando parte degli expert in RAM
DeepSeek-V4-Flash-Vision-Exp	284B / 13B MoE + vision	**Al limite**	Come sopra, ~168 GB, ancora più stretto
MiMo-V2.6-Flash	309B / 15B MoE	**Solo con offload**	In FP8 servono ~310 GB, che non entrano in VRAM. In Q4 (~160–170 GB) va tenendo una parte degli expert in RAM: funziona, ma è lento
GLM-5.3-Flash	320B / 18B MoE	**Solo con offload**	Stesso discorso di MiMo (~320 GB in FP8)
DeepSeek-V4.1-Flash	552B / 8–16B MoE	**Di fatto no**	Anche in Q4 sono ~300 GB, quindi la maggior parte va in RAM. Con 32 core darebbe pochi token al secondo. Meglio via API su Token Factory

Il collo di bottiglia per l'offload è la CPU. Quando gli expert stanno in RAM li calcola la CPU, e 32 core sono pochi. Per un batch sui referti, MiMo, GLM e V4.1 conviene usarli via API su Token Factory.

FP4 su H100: Hopper non calcola nativamente in FP4. I DeepSeek V4 entrano in VRAM solo se vLLM/SGLang dequantizzano al volo; convertiti in FP8 (~290 GB) non entrano.

Modello delle GPU: la scheda del nodo dice "96GB HBM2e", ma le H100 da ~94–96 GB sono di solito le NVL (HBM3). Non cambia i calcoli; *nvidia-smi* sul nodo dà il modello esatto.

Cosa cambierei nel setup attuale

1. Modelli densi da 27–31B in BF16 (Qwen3.8-27B, Gemma-4-31B, o MedGemma 27B): un salto netto di qualità rispetto a medgemma:4b-it-q4_K_M, che oggi usa meno del 5% di una GPU.

2. vLLM al posto di Ollama sopra i 30B: Ollama non supporta FP8 né il tensor parallel serio su 2 GPU; vLLM sì, con batching continuo che moltiplica il throughput sui batch di referti.

3. Risorse SLURM: per Qwen3.8-Flash-Next o i MoE in offload servono --gres=gpu:2, --mem intorno a 400G e --cpus-per-task=32 (oggi submit_h100.sh chiede gpu:1 e 64G).